// Day 29 of 30

CONCEPT + CODE

Circuit Breaker — Save Your Microservices

→ Swipe through all 6 slides

Shubham Bhati
linkedin.com/in/bhatishubham

The Problem

One slow service kills everything

```
Order Service → Payment Service

Payment Service is DOWN.

Order Service keeps calling...
  → Threads waiting...
  → Thread pool exhausted...
  → Order Service also dies.
  → User Service calls Order...
  → User Service also dies.

→ Cascading failure. Everything down.
```

💡 Without circuit breaker, one failure = total failure

Shubham Bhati
linkedin.com/in/bhatishubham

How Circuit Breaker Works

Like an electrical circuit breaker

```
Three states:

CLOSED (normal):
  → Requests pass through
  → Failures counted

OPEN (tripped):
  → Requests blocked immediately
  → Returns fallback response
  → No calls to failing service

HALF-OPEN (testing):
  → Allows few test requests
  → Success? → CLOSED
  → Fail? → OPEN again
```

💡 CLOSED → OPEN → HALF-OPEN → CLOSED (cycle)

Spring Boot Implementation

Resilience4j — 5 lines of code

```
@CircuitBreaker(
  name = "paymentService",
  fallbackMethod = "paymentFallback"
)
public PaymentResponse processPayment(
  PaymentRequest req) {
  return paymentClient.charge(req);
}

public PaymentResponse paymentFallback(
  PaymentRequest req, Exception e) {
  return new PaymentResponse("PENDING");
}
```

💡 Fallback = graceful degradation, not crash

Shubham Bhati
linkedin.com/in/bhatishubham

Configuration

Tune for your system

```
resilience4j.circuitbreaker:
  instances:
    paymentService:
      failure-rate-threshold: 50
      wait-duration-in-open-state: 30s
      sliding-window-size: 10

→ 50% failures in last 10 calls?
  → Circuit opens
→ Wait 30 seconds
→ Try again (half-open)
```

💡 Tune thresholds based on your SLAs

Other Resilience Patterns

Circuit breaker is just one tool

```
Retry:
  → Try again on transient failure
  → Max 3 retries, exponential backoff

Rate Limiter:
  → Max 100 requests per second

Bulkhead:
  → Isolate thread pools per service
  → One slow service can't starve others

Timeout:
  → Don't wait forever. Set limits.
```

💡 Production systems need ALL of these

Shubham Bhati
linkedin.com/in/bhatishubham

Interview Answer

When they ask about fault tolerance

```
Interviewer:
  'How do you handle service failures
   in microservices?'

You:
  'I use Resilience4j with multiple
   patterns:
   1. Circuit breaker for cascading fails
   2. Retry with exponential backoff
   3. Bulkhead for thread isolation
   4. Timeout to prevent hanging
   5. Fallback for graceful degradation'
```

💡 Knowing resilience patterns = senior-level signal

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham